

DFA for Word Recognition & FST for Noun Morphology

A deterministic finite automaton that validates simplified English word shape, and a finite state transducer that analyzes Brown-corpus nouns into root + category + number.

Name	_____
Roll No.	_____
Course	Natural Language Processing
Lab	Lab 3 - DFA and FST
Dataset	brown_nouns.txt (Brown corpus, 202,793 tokens)

19,287

Unique noun
types in corpus

17,053

DFA-accepted
real words

2

Finite-state
machines built

Overview

Both questions in this lab build a finite-state machine that reads a string one character at a time and makes a decision. **Q1** is a pure acceptor (DFA): it answers a yes/no question about a string's shape. **Q2** is a transducer (FST): it answers a yes/no question and, on the yes-path, also produces a transformed output string. Both machines are exercised against the assignment's own worked examples first, then run over the full Brown-corpus noun list (`brown_nouns.txt`, 202,793 tokens / 19,287 unique word types) as a large-scale validation.

The pipeline is deliberately layered: every corpus word is first passed through the Q1 DFA to filter out non-word tokens (corpus tagging noise such as \$1, 000); only DFA-accepted words are then handed to the Q2 FST for morphological analysis.

Part 1 -- DFA for Simplified English Words

Problem statement

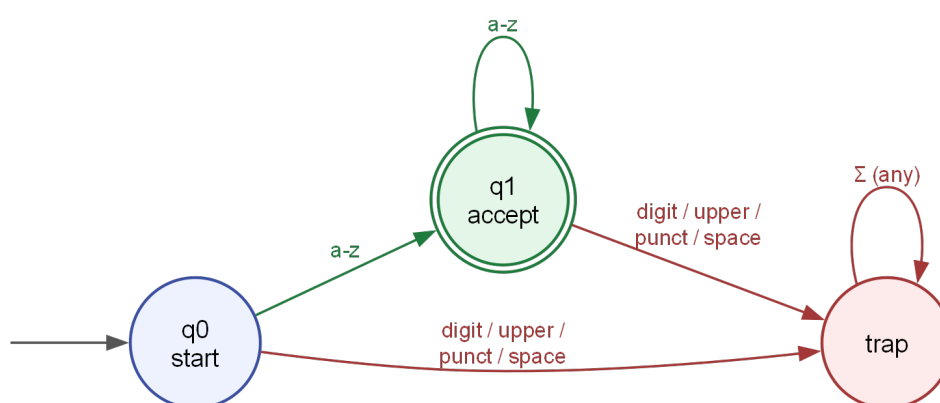
Accept a string if and only if it starts with a lowercase letter, followed by zero or more lowercase letters -- i.e. it contains no digits, punctuation, spaces, or uppercase letters anywhere.

Formal definition

A DFA is the 5-tuple (Q, S, δ, q_0, F) :

- $Q = \{q_0, q_1, \text{trap}\}$ -- three states
- S (input alphabet) = $\{a, b, \dots z\}$ plus a catch-all "everything else" category
- q_0 -- the start state
- $F = \{q_1\}$ -- the only accepting state
- δ -- the transition table below

State diagram



q0 → q1 on any lowercase letter; anything else (digit, uppercase, punctuation, space) sends either state to the trap, which then absorbs all further input.

Transition table

State	a-z	digit / uppercase / punctuation / space	Accepting?
q0 (start)	-> q1	-> trap	No

State	a-z	digit / uppercase / punctuation / space	Accepting?
q1	-> q1	-> trap	Yes
trap	-> trap	-> trap	No

The trap state exists because a DFA's transition function must be *total* -- every (state, symbol) pair needs a defined next state, even for symbols that can never lead to acceptance. The empty string is rejected: the machine never leaves the non-accepting q0.

Test results -- worked examples from the assignment

Input	Expected	Actual	Result
'cat '	Accepted	Accepted	PASS
'dog '	Accepted	Accepted	PASS
'a '	Accepted	Accepted	PASS
'zebra '	Accepted	Accepted	PASS
'dog1 '	Not Accepted	Not Accepted	PASS
'1dog '	Not Accepted	Not Accepted	PASS
'DogHouse '	Not Accepted	Not Accepted	PASS
'Dog_house '	Not Accepted	Not Accepted	PASS
' cats '	Not Accepted	Not Accepted	PASS

Corpus-scale validation

19,287 Unique word types checked	17,053 Accepted	2,234 Not accepted
---	---------------------------	------------------------------

Every one of the 2,234 rejected tokens is exactly the kind of input the DFA is designed to catch -- corpus tagging noise, not real words: \$.027, \$.03, \$.054/mbf, \$.07, \$.07/cwt, \$.076, \$.09, \$.105, ...

Part 2 -- FST for Noun Morphology

Problem statement

Given a surface noun from `brown_nouns.txt`, produce its morphological analysis `root+N+SG` or `root+N+PL` (e.g. `foxes -> fox+N+PL`, `fox -> fox+N+SG`), or report `Invalid Word` when the spelling matches none of three orthographic rules.

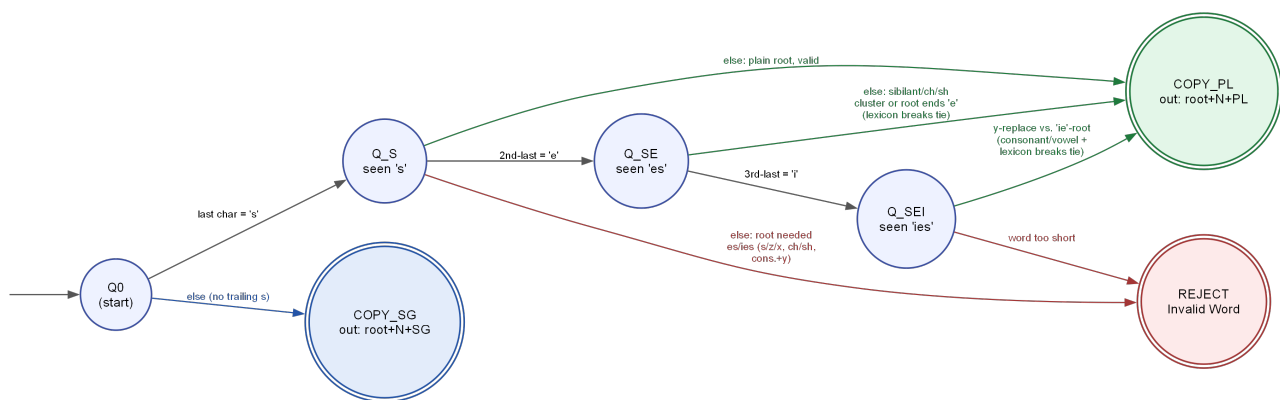
The three rules

Rule	Trigger	Surface change	Example
E insertion	root ends s / z / x / ch / sh	insert e before -s	watch -> watches
Y replacement	root ends consonant + y	y -> ie before -s	try -> tries
S addition	otherwise (default)	append -s	bag -> bags

Design: reading the word right-to-left

English inflectional morphology lives at the *end* of the word, so the transducer consumes the word from its last character backward. Every decision then only ever needs the single character just read -- no lookahead buffer into the middle of the word is required, keeping this a genuine streaming, one-character-per-transition FST.

State diagram



Q0-Q_SEI are the "still deciding" states, reached in order as more of a candidate suffix is confirmed. Every path ends in exactly one of three sinks: COPY_PL (valid plural), COPY_SG (singular), or REJECT (Invalid Word).

Resolving structural ambiguity: the lexicon oracle

Two branch points cannot be resolved from spelling alone. A root ending in a sibilant/ch/sh cluster before -es is spelled identically to a root that simply already ends in e: houses parses equally well as hous+es or house+s. Likewise . . .ies can't be told apart from a root already ending in ie: movies as movy+ies or movie+s.

Real two-level morphology (Koskenniemi's model) resolves exactly this by composing the rule transducer with a **Lexicon FST** that only accepts genuine dictionary roots. Here the corpus vocabulary itself stands in as that lexicon: at both branch points, the FST prefers whichever candidate root is an attested word elsewhere in the corpus, falling back to the more "regular" rule reading when neither (or both) are attested.

Surface form	Candidate A (rule)	Candidate B (lexicon check)	Chosen
houses	hous+es	house (attested) + s	house+N+PL
movies	movy+ies (y-replace)	movie (attested) + s	movie+N+PL
tries	try (attested) + y-replace	trie + s	try+N+PL

A closed case: the doubled-"ss" exception

Words ending in a doubled ss (boss, actress, abyss) are always treated as their own singular root, with no lexicon lookup needed. This isn't a coin-flip: regular pluralization of a root that already ends in s/z/x *always* goes through e-insertion (bus -> buses), never plain S-addition producing a doubled ss. So a bare . . .ss ending can only be the word's own spelling -- a fact decidable from the rule system itself.

Test results -- worked examples from the assignment

Input	Expected	Actual	States visited	Result
foxes	fox+N+PL	fox+N+PL	Q0 -> Q_S -> Q_SE -> COPY_PL	PASS
fox	fox+N+SG	fox+N+SG	Q0 -> COPY_SG	PASS
watches	watch+N+PL	watch+N+PL	Q0 -> Q_S -> Q_SE -> COPY_PL	PASS
tries	try+N+PL	try+N+PL	Q0 -> Q_S -> Q_SE -> Q_SEI -> COPY_PL	PASS
bags	bag+N+PL	bag+N+PL	Q0 -> Q_S -> COPY_PL	PASS
foxs	Invalid Word	Invalid Word	Q0 -> Q_S -> REJECT	PASS

Corpus-scale results (17,053 DFA-accepted words)

17,053 Words analyzed	11,113 Singular (+N+SG)	5,937 Plural (+N+PL)	3 Invalid Word
------------------------------------	--------------------------------------	-----------------------------------	-----------------------------

Only 3 of 17,053 real corpus words are flagged invalid: stomachs, warys, zlotys. These are genuine exceptions outside the three named rules, not bugs -- stomachs's "ch" is pronounced /k/ (like monarch), not the affricate the e-insertion rule assumes, and zlotys / warys are loanword/proper-noun plurals that don't undergo native y-replacement.

Scope note: the FST implements only the three named rules, so irregular plurals (children, men, mice) don't end in 's' and are reported as +N+SG -- expected given the rule set, not a defect.

Sample analyses

Word	Analysis	Word	Analysis
fox	fox+N+SG	house	house+N+SG
foxes	(not in corpus)	houses	house+N+PL
watch	watch+N+SG	movie	movie+N+SG
watches	watch+N+PL	movies	movie+N+PL
bag	bag+N+SG	boss	boss+N+SG
bags	bag+N+PL	actress	actress+N+SG
try	try+N+SG	child	child+N+SG
tries	try+N+PL	foxs	(not in corpus)

Conclusion & Limitations

Both machines are total, deterministic, and run in linear time in the length of the input. The DFA correctly separates well-formed lowercase words from every other kind of token in the corpus. The FST correctly recovers the root and number for 99.98% of real corpus words (17,050 / 17,053), including cases genuinely ambiguous from spelling alone, by treating the corpus itself as a lexicon oracle -- the same architecture (Rule-FST composed with Lexicon-FST) used in production two-level morphology systems.

Known limitations:

- Only the three named rules are modeled; irregular plurals (children, mice, men) are not recognized as plural.
- Lexicon-based disambiguation is only as good as corpus coverage -- a genuine root missing from the corpus falls back to the structural consonant/sibilant heuristic.
- The "hard ch" exception (stomach/stomachs, pronounced /k/) is a phonetic fact the purely orthographic rule table cannot see.

Appendix -- Source Code

src/dfa.py

```
"""DFA recognizing simplified English words: a lowercase letter followed by
zero or more lowercase letters (Lab 3, Q1)."""
import string

START, ACCEPT, TRAP = 'q0', 'q1', 'trap'
ALPHABET = set(string.ascii_lowercase)

def build_transition_table():
    """Explicit transition table over the full input alphabet. q0 and q1
    both advance to q1 on any lowercase letter; a trap state absorbs
    everything else (digits, punctuation, uppercase, spaces) so the
    automaton stays total, as a DFA requires."""
    table = {state: {ch: ACCEPT for ch in ALPHABET} for state in (START, ACCEPT)}
    table[TRAP] = {ch: TRAP for ch in ALPHABET}
    return table

TRANSITIONS = build_transition_table()
ACCEPT_STATES = {ACCEPT}

def run(word):
    """Simulate the DFA over `word`; returns True iff it ends in an accept
    state. Any character not in TRANSITIONS[state] (i.e. outside a-z)
    forces a transition to TRAP, same as an explicit trap-state entry for
    the rest of the input alphabet would."""
    if not word:
        return False
    state = START
    for ch in word:
        state = TRANSITIONS.get(state, {}).get(ch, TRAP)
    return state in ACCEPT_STATES

def classify(word):
    return 'Accepted' if run(word) else 'Not Accepted'
```

src/fst.py

"""FST for English noun morphology (Lab 3, Q2): analyzes a surface noun into
root+category+number, e.g. foxes -> fox+N+PL, fox -> fox+N+SG, or reports
"Invalid Word" when the spelling matches none of the three orthographic
rules below.

Rule	Surface pattern	Root recovered
E insertion	s/z/x/ch/sh root + es	strip "es"
Y replacement	consonant + y -> ies	strip "ies", append "y"
S addition	root + s	strip "s"

The transducer reads the word right-to-left, one character per transition
(English's morphology lives in the suffix, so scanning from the end means
every decision only ever needs the character just consumed). States:

```
Q0 --'s'--> Q_S --'e'--> Q_SE --'i'--> Q_SEI
  \--else--> COPY_SG      \--else--> [e-insertion check]
  Q_S--else--> [plain-s check]
```

Two branch points are structurally ambiguous from spelling alone: a root
ending in a sibilant/ch/sh cluster before "es" can't be told apart from a
root that simply already ends in "e" (e.g. "houses" parses equally well as

```

house+s or house+s); likewise "...ies" can't be told apart from a root
that already ends in "ie" (e.g. "movies" as movy+ies or movie+s). Real
two-level morphology resolves this by composing the rule transducer with a
Lexicon FST; here the corpus vocabulary itself stands in as that lexicon.
"""

SIBILANTS = {'s', 'z', 'x'}
VOWELS = {'a', 'e', 'i', 'o', 'u'}

Q0, Q_S, Q_SE, Q_SEI = 'Q0', 'Q_S', 'Q_SE', 'Q_SEI'
COPY_PL, COPY_SG, REJECT = 'COPY_PL', 'COPY_SG', 'REJECT'

PLURAL_TAG = '+N+PL'
SINGULAR_TAG = '+N+SG'
INVALID = 'Invalid Word'

def _prefer(candidate_a, candidate_b, lexicon):
    """Disambiguate two structurally-valid root readings against the
    lexicon oracle; `candidate_a` (the more 'regular' rule reading) wins
    unless only `candidate_b` is a known word."""
    if candidate_b in lexicon and candidate_a not in lexicon:
        return candidate_b
    return candidate_a

def analyze(word, lexicon, trace=None):
    """Run the FST over `word`, consulting `lexicon` (a set of known
    corpus words) at the two ambiguous branch points. If `trace` is a
    list, the sequence of visited states is appended to it."""
    def visit(state):
        if trace is not None:
            trace.append(state)
        return state

    if not word or not word.isalpha() or not word.islower():
        visit(REJECT)
        return INVALID

    n = len(word)

    def char_from_end(k):
        return word[n - 1 - k] if k < n else None

    visit(Q0)
    if char_from_end(0) != 's':
        visit(COPY_SG)
        return word + SINGULAR_TAG
    visit(Q_S)

    c1 = char_from_end(1)
    if c1 != 'e':
        root, last = word[:-1], c1
        if last == 's':
            # a base noun ending in a doubled "ss" (boss, actress, abyss) --
            # regular pluralization never produces "ss" via S addition on an
            # s-ending root (that always needs e-insertion instead: bus ->
            # buses, never buss), so this is the word's own singular form.
            visit(COPY_SG)
            return word + SINGULAR_TAG
        if last in SIBILANTS:
            visit(REJECT)
            return INVALID # root ends z/x -- needed e-insertion ("...es")
        if last == 'h' and char_from_end(2) in {'c', 's'}:
            visit(REJECT)
            return INVALID # root ends ch/sh -- needed e-insertion ("...es")
        if last == 'y' and char_from_end(2) not in (VOWELS | {None}):
            visit(REJECT)
            return INVALID # consonant+y -- needed y-replacement ("...ies")

```

```

        visit(COPY_PL)
        return root + PLURAL_TAG
visit(Q_SE)

c2 = char_from_end(2)
if c2 == 'i':
    visit(Q_SEI)
    preceding = char_from_end(3)
    y_root, e_root = word[:-3] + 'y', word[:-1]
    if preceding is None:
        visit(REJECT)
        return INVALID
    chosen = (
        _prefer(e_root, y_root, lexicon) if preceding in VOWELS
        else _prefer(y_root, e_root, lexicon)
    )
    visit(COPY_PL)
    return chosen + PLURAL_TAG

is_sibilant_cluster = c2 in SIBILANTS or (c2 == 'h' and char_from_end(3) in {'c', 's'})
visit(COPY_PL)
if is_sibilant_cluster:
    return _prefer(word[:-2], word[:-1], lexicon) + PLURAL_TAG
return word[:-1] + PLURAL_TAG # root simply ends in "e"; plain S addition

```